

Aero 405: Biconvex Airfoil Project

Zach Selzman

05/11/2026

Contents

1. Lift C_l	1
2. Drag C_d	2
3. Moment C_m	3
4. Discussion	3
5. Code	4
Bibliography	9

1. Lift C_l

It is worth noting that the report uses C_p to denote what we in class call C_l , and likewise C_r for C_d . Moreover, for the moment coefficient C_m , the experimental data took the moment coefficient about the leading edge [1].

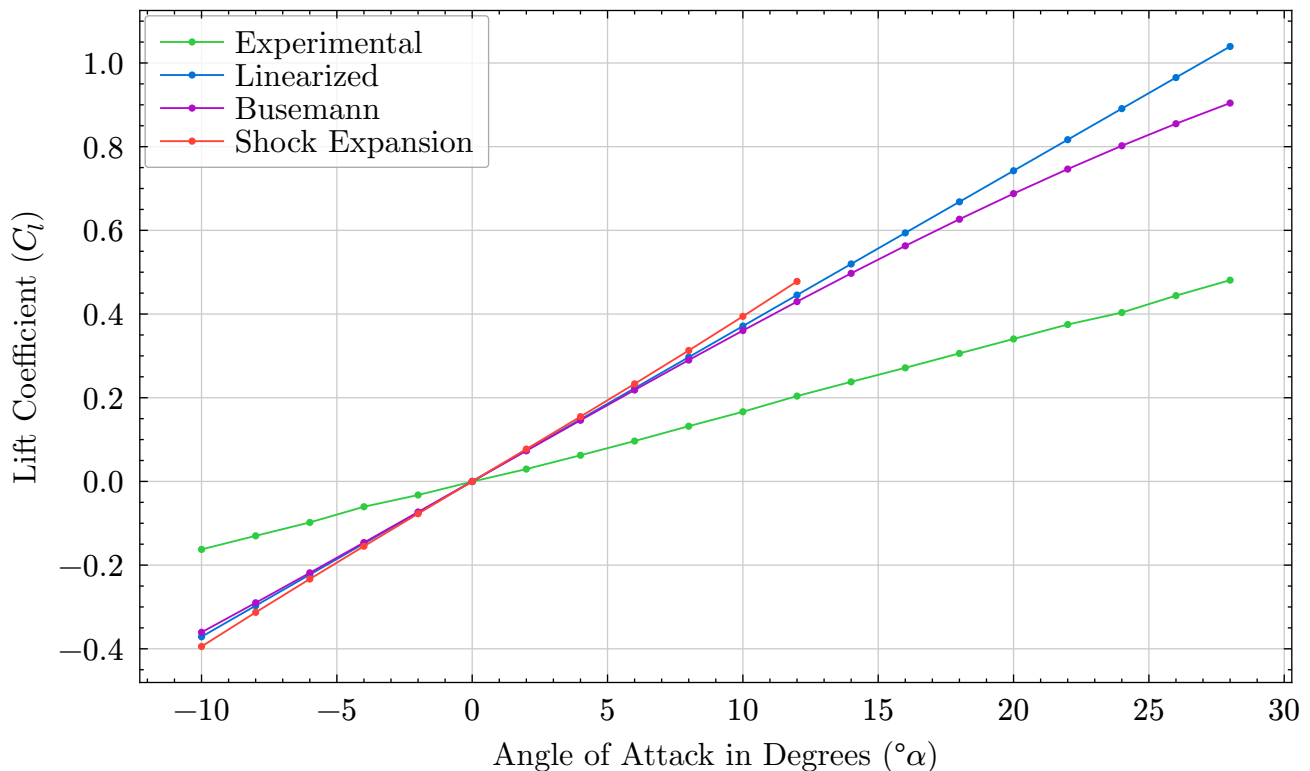


Figure 1: GU2 lift coefficient vs. angle of attack

Note, in Figure 1, the Shock-Expansion is only plotted for $10 \leq \alpha \leq 12^\circ$. This is because at high angles of attack, the shock wave becomes detached from the GU2 surface, forming something similar to a bow shock (this can be observed in the schlieren images in [1]). When the flow is subsonic after the shock, I assumed that the shock was no longer attached to the surface (the strong solution). As such, shock expansion theory is no longer valid outside of the small angle range. Furthermore, both Linearization and Busemann methods have small angle approximations baked into them, either through the assumption of small perturbations (inherently implying small angles), or the neglect of higher order terms.

2. Drag C_d

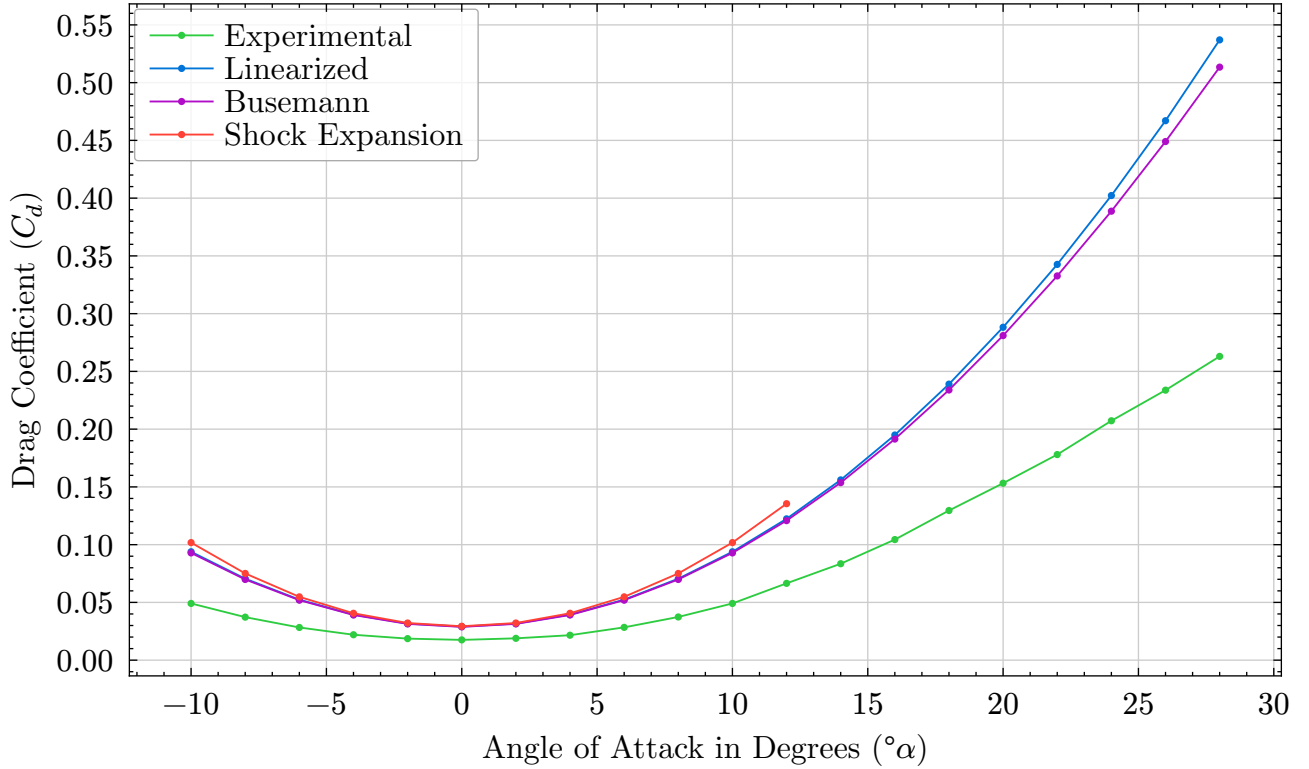


Figure 2: GU2 drag coefficient vs. angle of attack

The same limitations apparent for Figure 2 are the same in Figure 1, where the detached shock at high angles of attack renders it physically invalid.

3. Moment C_m

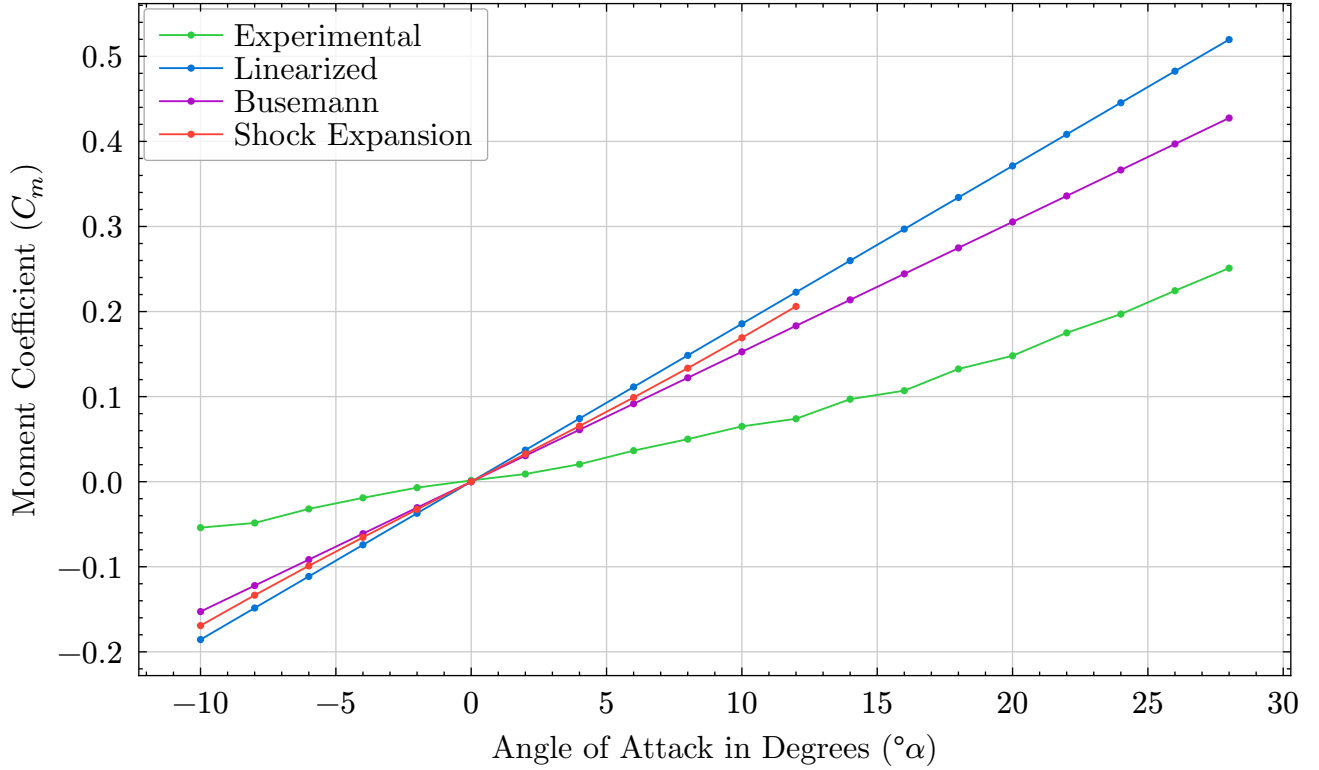


Figure 3: GU2 moment coefficient vs. angle of attack

For the moment coefficient, a positive moment implies that there is an upward pitching moment on the airfoil about the leading edge. The same limitations apparent for Figure 3 are the same in Figure 1 and Figure 2, where the detached shock at high angles of attack renders it physically invalid in the degree range.

4. Discussion

All plots generally feature the same trends, where the experimental data reveals over-predictions by the theoretical analysis. However, there is consistent convergence between the three analytical methods. Busemann, being the higher-order approximation of the Small-Perturbation method, is more accurate for all coefficients. Interestingly (and perhaps reflective of a mistake), the Shock-Expansion method is less accurate compared to the approximate methods, despite taking into the physical phenomena that is occurring (shocks and expansion waves). One observation worth noting is that all data was virtually equivalent at zero degrees of attack ($\alpha = 0^\circ$), where C_m , C_l , and C_d all converged towards a singular value. The one exception to this was in Figure 2, where the experimental C_d at $\alpha = 0^\circ$ was just slightly lower than the linearization, busemann,

and shock-expansion methods. However, the overall shape of each plot was fairly uniform when comparing methodology, so I consider this to largely be successful.

5. Code

Matlab

```

1 % I want to do my plots in typst, so this is just to get the values and
2 % perform the calculations. There will be four lines on each plot, and
3 % three plots in total: C_l, C_d, and C_m. So, 12 sets of data are needed.
4 %% Experimental Data
5 % an angle range is provided, I'm taking the average
6 alpha_e=[-10,-8,-6,-4,-2,0,2,4,6,8,10,12,14,16,18,20,22,24,26,28];
7 % 100* C_l
8 Cl_e = [-16.25, -13.0, -9.8, -6.05, -3.25, -0.05, 2.95, ...
9         6.25, 9.65, 13.2, 16.65, 20.4, 23.8, 27.15, ...
10        30.6, 34.05, 37.5, 40.35, 44.4, 48.1];
11 Cd_e=[4.91, 3.725, 2.83, 2.20, 1.865, 1.755, 1.89, ...
12       2.16, 2.84, 3.74, 4.91, 6.65, 8.35, 10.44, ...
13       12.955, 15.32, 17.8, 20.725, 23.375, 26.30];
14 Cm_e=[-5.4, -4.85, -3.2, -1.9, -0.7, 0.15, 0.9, ...
15       2.05, 3.65, 5.0, 6.5, 7.4, 9.7, 10.7, ...
16       13.25, 14.8, 17.5, 19.7, 22.45, 25.1];
17 Cl_e = Cl_e/100;
18 Cd_e = Cd_e/100;
19 Cm_e = Cm_e/100;
20 %% Linear Supersonic Small Disturbance
21 alpha=alpha_e*pi/180;
22 M=2.13;
23 B=sqrt(M^2-1);
24 Cl_l=4.* alpha./B;
25 % for drag, the GU2 is symmetric so we only care about alpha and thickness
26 x = linspace(0,1,1000);
27 R = 2.5;
28 %thickness expression
29 yt = sqrt(R^2 - (x - 0.5).^2) - sqrt(R^2 - 0.5^2); % the expression derived
    from the provided circle [Ferri]
30 yc = zeros(size(x));
31 %upper and lower surfaces
32 yu = yc + yt;
33 yl = yc - yt;
34 dyt_dx = gradient(yt,x);
35 dyc_dx = gradient(yc,x);
36 % thickness integrals (Im assuming that c=1.
37 I_thick = trapz(x,dyt_dx.^2);
38 Cd_l=(4/B)*(alpha.^2+I_thick);
39 % for moment, only alpha at play
40 Cm_l=(4/B)*(alpha./2);

```

```

41
42 %% Busemann Method
43 gamma = 1.4;
44 A = 2/B;
45 K = ((gamma+1)*M^4 - 4*M^2 + 4)/(2*(M^2 - 1)^2);
46 % top and bottom surface slopes
47 dyu_dx = gradient(yu,x);
48 dyl_dx = gradient(yl,x);
49 theta_u = atan(dyu_dx); % deflections
50 theta_l = atan(dyl_dx);
51
52 Cl_b = zeros(size(alpha));
53 Cd_b = zeros(size(alpha));
54 Cm_b = zeros(size(alpha));
55
56 for j = 1:length(alpha)
57
58     a = alpha(j);
59     % compression-delta is positive
60     delta_u = theta_u - a;
61     delta_l = a - theta_l;
62     % Busemann Cps
63     Cp_u = A*delta_u + K*delta_u.^2;
64     Cp_l = A*delta_l + K*delta_l.^2;
65
66     % forces yo
67     CN = trapz(x, Cp_l - Cp_u);
68     CA = trapz(x, Cp_u.*dyu_dx - Cp_l.*dyl_dx);
69
70     % use forces to get lift and drag
71     Cl_b(j) = CN*cos(a) - CA*sin(a);
72     Cd_b(j) = CN*sin(a) + CA*cos(a);
73
74     % conventional moment about leading edge
75     Cm_conv = trapz(x, x.*(Cp_l - Cp_u));
76 end
77 %% Shock Expansion Method (ah hell)
78
79 dx = diff(x); % slopes of the top and bottom surfaces
80 dyu = diff(yu);
81 dyl = diff(yl);
82
83 xmid = 0.5*(x(1:end-1) + x(2:end)); % need to break the GU2 up into panels,
    basically just assume it is a ton of flat plates
84
85 theta_u_pan = atan2(dyu,dx);

```

```

86 theta_l_pan = atan2(dyl,dx); % deflection angle, same as for Busemann
87
88 Cl_se = NaN(size(alpha));
89 Cd_se = NaN(size(alpha));
90 Cm_se = NaN(size(alpha));
91
92 for j = 1:length(alpha)
93     a = alpha(j);
94     % compute the shock expansion pressures for the top and bottom surfaces
95     % the function is below
96     [Cp_u, ok_u] = shockExpansionSurface(theta_u_pan, "upper", a, M,
gamma);
97     [Cp_l, ok_l] = shockExpansionSurface(theta_l_pan, "lower", a, M,
gamma);
98
99     if ~(ok_u && ok_l)
100         continue;
101     end
102     % integrate pressures to get forces
103     CN = sum((Cp_l - Cp_u).*dx);
104     CA = sum(Cp_u.*dyu - Cp_l.*dyl);
105     % use forces to get lift, drag and moment.
106     Cl_se(j) = CN*cos(a) - CA*sin(a);
107     Cd_se(j) = CN*sin(a) + CA*cos(a);
108     Cm_se(j) = sum(xmid.*(Cp_l - Cp_u).*dx);
109 end
110 % Output table for Typst
111 T = table(alpha_e(:), ...
112     Cl_e(:), Cl_l(:), Cl_b(:), Cl_se(:), ...
113     Cd_e(:), Cd_l(:), Cd_b(:), Cd_se(:), ...
114     Cm_e(:), Cm_l(:), Cm_b(:), Cm_se(:), ...
115     'VariableNames', {'alpha_deg', ...
116     'Cl_exp', 'Cl_linear', 'Cl_busemann', 'Cl_shockexp', ...
117     'Cd_exp', 'Cd_linear', 'Cd_busemann', 'Cd_shockexp', ...
118     'Cm_exp', 'Cm_linear', 'Cm_busemann', 'Cm_shockexp'});
119
120 disp(T);
121 writetable(T, 'GU2_results.csv');
122
123 % Local Functions
124
125 function [Cp, ok] = shockExpansionSurface(theta, side, alpha, M, gamma)
126
127     n = length(theta);
128     Cp = NaN(1,n);
129     ok = true;

```

```

130
131     M_local = M;
132     p_ratio = 1.0;      % p_local / p_inf
133
134     % This effective reference angle makes:
135     % upper leading-edge compression angle = theta_u - alpha
136     % lower leading-edge compression angle = alpha - theta_l
137     theta_old = alpha;
138
139     for i = 1:n
140         theta_new = theta(i);
141
142         if side == "upper"
143             delta = theta_new - theta_old;
144         else
145             delta = theta_old - theta_new; % use symmetry here
146         end
147
148         if abs(delta) < 1e-12
149             p2_p1 = 1.0;
150             M2 = M_local; % if there is no deflection, we're basically in
the free stream yo
151
152         elseif delta > 0
153             % Compression: attached oblique shock
154             [M2, p2_p1, attached] = obliqueShockWeak(M_local, delta,
gamma);
155
156             if ~attached
157                 ok = false;
158                 return;
159             end
160
161         else
162             % Expansion: Prandtl-Meyer fan
163             [M2, p2_p1] = prandtlMeyerExpansion(M_local, abs(delta),
gamma);
164         end
165
166         p_ratio = p_ratio*p2_p1;
167
168         Cp(i) = (p_ratio - 1)/(0.5*gamma*M^2); % Cp equation from notes
169
170         M_local = M2;
171         theta_old = theta_new;
172     end

```

```

173 end
174
175 function [M2, p2_p1, attached] = obliqueShockWeak(M1, delta, gamma)
176
177     attached = true;
178
179     if M1 <= 1
180         attached = false;
181         M2 = NaN;
182         p2_p1 = NaN;
183         return;
184     end
185
186     beta_min = asin(1/M1) + 1e-8;
187     beta_max = pi/2 - 1e-8;
188
189     beta_grid = linspace(beta_min,beta_max,2000);
190     theta_grid = thetaBetaM(M1,beta_grid,gamma);
191
192     theta_max = max(theta_grid);
193
194     if delta > theta_max
195         attached = false;
196         M2 = NaN;
197         p2_p1 = NaN;
198         return;
199     end
200
201     f = @(b) thetaBetaM(M1,b,gamma) - delta;
202
203     % Weak shock root lies between Mach angle and beta at theta_max
204     [~,idx_max] = max(theta_grid);
205     beta_upper = beta_grid(idx_max);
206
207     beta = fzero(f,[beta_min,beta_upper]); % solve for roots, alternative
to theta-beta mach plot
208     % oblique gas dynamics
209     Mn1 = M1*sin(beta);
210
211     p2_p1 = 1 + 2*gamma/(gamma+1)*(Mn1^2 - 1);
212
213     Mn2_sq = (1 + 0.5*(gamma-1)*Mn1^2)/(gamma*Mn1^2 - 0.5*(gamma-1));
214     Mn2 = sqrt(Mn2_sq);
215
216     M2 = Mn2/sin(beta - delta);
217 end

```



```

218
219 function theta = thetaBetaM(M,beta,gamma)
220
221     numerator = 2*cot(beta).*(M^2.*sin(beta).^2 - 1);
222     denominator = M^2*(gamma + cos(2*beta)) + 2;
223
224     theta = atan(numerator./denominator);
225 end
226
227 function [M2, p2_p1] = prandtlMeyerExpansion(M1, delta, gamma)
228
229     nu1 = prandtlMeyerNu(M1,gamma);
230     nu2 = nu1 + delta;
231
232     f = @(M) prandtlMeyerNu(M,gamma) - nu2;
233
234     M2 = fzero(f,[M1,50]);
235
236     p2_p1 = ((1 + 0.5*(gamma-1)*M1^2)/(1 + 0.5*(gamma-1)*M2^2))^(gamma/(gamma-1));
237 end
238
239 function nu = prandtlMeyerNu(M,gamma)
240
241     nu = sqrt((gamma+1)/(gamma-1))*atan(sqrt((gamma-1)/(gamma+1)*(M^2 - 1))) ...
242         - atan(sqrt(M^2 - 1));
243 end

```

Bibliography

- [1] A. Ferri, “Experimental Results with Airfoils Tested in the High-speed Tunnel at Guidonia,” Technical Memorandum NACA-TM-946, Jul. 1940. [Online]. Available: <https://ntrs.nasa.gov/api/citations/19930094471/downloads/19930094471.pdf>